

Project Title

Detection of SSH Brute-Force Attack using Splunk SIEM

Objective

The objective of this project is to simulate an SSH brute-force attack in a controlled lab environment and implement detection mechanisms using a SIEM solution. The project focuses on identifying unauthorized access attempts by analyzing authentication logs and correlating multiple events to detect malicious behavior.

Lab Environment Setup

- **Attacker Machine:** Kali Linux
 - **Target Machine:** Metasploitable
 - **SIEM Tool:** Splunk Enterprise
 - **Virtualization Platform:** Oracle VirtualBox
 - **Network Configuration:** NAT Network (same subnet communication)
-

Network Verification

To ensure connectivity between attacker and target:

```
ping <target_ip>
```

Purpose:

Verify network communication between systems before launching attack.

Attack Simulation using Metasploit

The SSH brute-force attack was performed using the Metasploit.

Step 1: Launch Metasploit

```
msfconsole
```

Step 2: Load SSH brute-force module

```
use auxiliary/scanner/ssh/ssh_login
```

Step 3: Configure target and credentials

```
set RHOSTS <target_ip>
set USERNAME msfadmin
set PASSWORD wrongpass
```

Step 4: Execute attack

```
run
```

Result:

- Multiple failed login attempts generated in logs
-

Step 5: Successful Login Attempt

```
set PASSWORD msfadmin  
run
```

Result:

- Successful login achieved
 - SSH session established
-

Log Monitoring on Target System**Log File Used**

/var/log/auth.log

Command

```
sudo tail -f /var/log/auth.log
```

Observed Events

- Failed password attempts
 - Accepted password (successful login)
 - SSH session activity
-

Post-Exploitation Activity (Persistence Simulation)

```
sudo adduser hackersuser
```

Purpose:

Simulate attacker persistence by creating a new user account.

Log Collection and Transfer

Logs were transferred from the target machine to the SIEM system:

```
scp -o HostKeyAlgorithms=+ssh-rsa msfadmin@<target_ip>:~/auth.log .
```

Purpose:

- Securely transfer logs
 - Prepare data for ingestion into Splunk
-

Log Ingestion into Splunk

Steps:

- Open Splunk Web Interface
 - Navigate to **Settings** → **Add Data** → **Upload**
 - Upload auth.log
 - Set index as main
-

Log Verification in Splunk

```
index=main "Failed password"
```

```
index=main "Accepted password"
```

Purpose:

Confirm successful ingestion of logs.

Detection Logic Implementation

SPL Query

```
index=main ("Failed password" OR "Accepted password")
| rex "from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| stats count(eval(searchmatch("Failed password"))) as failed,
          count(eval(searchmatch("Accepted password"))) as success
          by src_ip
| where failed > 5 AND success > 0
```

This query identifies brute-force attacks by correlating multiple failed login attempts followed by a successful login from the same source IP.

Detection Explanation

This query performs:

- Extraction of source IP addresses
- Counting failed login attempts
- Counting successful logins
- Correlating events to detect brute-force success

Detection Condition:

- Multiple failed attempts from same IP
- Followed by successful login
- Indicates **account compromise**

Alert Configuration

- Saved query as alert
- Trigger condition: Number of results > 0
- Time range: Last 15 minutes

Purpose:

Enable real-time detection of brute-force attacks.

Visualization

- Created bar chart in Splunk
- X-axis: Source IP
- Y-axis: Number of attempts

Outcome:

Clear identification of attacker IP and attack intensity

Results

- Detected brute-force attack using correlation-based analysis
 - Identified attacker IP (192.168.x.x)
 - Observed unauthorized access through successful authentication
 - Simulated persistence by creating a new user
 - Implemented real-time alerting in Splunk
-

Key Skills Demonstrated

- SIEM (Splunk)
 - Log Analysis
 - Threat Detection
 - Incident Investigation
 - Attack Simulation
 - Security Monitoring
-

Use Case / Real-World Relevance

This detection approach is commonly used in SOC environments to identify brute-force attacks on SSH services. It helps analysts detect unauthorized access attempts and respond to potential security incidents in real time.

This project demonstrates the practical implementation of SOC workflows by simulating real-world attack scenarios and building detection mechanisms. It highlights the importance of event correlation in identifying security incidents rather than relying on individual log entries.

```
Apr 24 10:36:04 metasploitable login[4656]: pam_unix(login:session): session opened for user msfadmin by LOGIN(uid=0)
Apr 24 10:39:01 metasploitable CRON[4764]: pam_unix(cron:session): session opened for user root by (uid=0)
Apr 24 10:39:02 metasploitable CRON[4764]: pam_unix(cron:session): session closed for user root
Apr 24 10:43:33 metasploitable sshd[4783]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.103 user=msfadmin
Apr 24 10:43:36 metasploitable sshd[4783]: Failed password for msfadmin from 192.168.56.103 port 34785 ssh2
Apr 24 10:44:02 metasploitable sshd[4785]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.103 user=msfadmin
Apr 24 10:44:04 metasploitable sshd[4785]: Failed password for msfadmin from 192.168.56.103 port 37955 ssh2
Apr 24 10:44:25 metasploitable sshd[4787]: Accepted password for msfadmin from 192.168.56.103 port 41673 ssh2
Apr 24 10:44:25 metasploitable sshd[4789]: pam_unix(sshd:session): session opened for user msfadmin by (uid=0)
Apr 24 10:47:20 metasploitable sudo: msfadmin : TTY=tty1 ; PWD=/home/msfadmin ; USER=root ; COMMAND=/usr/bin/tail -f /var/log/auth.log
Apr 24 10:47:20 metasploitable sudo: pam_unix(sudo:session): session opened for user root by msfadmin(uid=0)
Apr 24 10:47:20 metasploitable sudo: pam_unix(sudo:session): session closed for user root
```

splunk>enterprise Apps ▾

Search Analytics Datasets Reports Alerts Dashboards Modules

Search & Reporting

New Search

Save As ▾ Create Table View Close

Time range: All time 🔍

```
index=main ("Failed password" OR "Accepted password")
| rex 'from (?src_ip\d+\.\d+\.\d+\.\d+)'
| stats count(eval(searchmatch("Failed password"))) as failed,
count(eval(searchmatch("Accepted password"))) as success
  by src_ip
| where failed > 5 AND success > 0
```

81 events (before 4/24/26 8:27:37:000 PM) No Event Sampling ▾

Job ▾ || ▹ ▸ ↻ ⚙ Smart Mode ▾

Events Patterns Statistics 🔍 Visualization

Chart: Column Chart ▾ Format ▾ 13 Trellis ▾

src_ip	failed	success
192.168.56.103	6	5